RELATÓRIO TÉCNICO DE DESENVOLVIMENTO DE SOFTWARE

Curso Técnico em Desenvolvimento de Sistemas | ETEC / CPS

Componente: Desenvolvimento de Sistemas I — Agenda 11 (Estruturas de Dados e colorama)

Estudante: Iraci Berto Gomes | **Arquivo de Entrega:** IraciBertoGomes_Ag11_DSI

1. Cenário e Objetivo da Atividade

No papel de desenvolvedor contratado para automatizar a telemetria de um reservatório de água, concebi uma solução procedural em Python executada no terminal. O sistema tem como objetivo emitir alertas visuais com cores distintas para **5 níveis progressivos de criticidade**, permitindo ao operador identificar rapidamente situações de risco de desabastecimento ou transbordamento. Para o enriquecimento cromático, utilizou-se a biblioteca **colorama**, com restauração explícita das configurações do terminal ao término de cada mensagem.

2. Tabela Oficial de Níveis, Situações e Cores do Reservatório

Nível	Situação Operacional	Cor Sugerida	Código Colorama	Diagnóstico Técnico
Nível 1	Muito baixo (crítico)	Vermelho	`Fore.RED`	Válvulas abertas. Risco iminente de desabastecimento.
Nível 2	Baixo	Amarelo	`Fore.YELLOW`	Bomba secundária acionada. Atenção para reposição.
Nível 3	Médio	Verde	`Fore.GREEN`	Capacidade estável. Volume ideal de operação.
Nível 4	Alto	Ciano	`Fore.CYAN`	Margem ampla. Entrada de água operando em 50%.
Nível 5	Muito alto (alerta)	Azul	`Fore.BLUE`	Capacidade máxima! Vertedouro ativado contra transbordo.

3. Arquitetura Modular: Funções e Procedimentos

Identificador	Classificação	Parâmetros	Retorno	Responsabilidade Principal
`limpar_tela`	Procedimento	*Nenhum*	`None`	Limpa o console multiplataforma de forma segura.
`exibir_cabecalho`	Procedimento	*Nenhum*	`None`	Imprime cabeçalho visual estilizado no terminal.
`definir_cor_nivel`	Função pura	`nivel (int)`	`str (ANSI)`	**Mapeia o nível para a cor da colorama (SRP).**
`gerar_barra_nivel`	Função utilitária	`nivel, max`	`str`	Renderiza barra gráfica ASCII de telemetria.
`exibir_alerta_nivel`	Procedimento	`item (dict)`	`None`	Exibe alerta colorido e restaura com Style.RESET_ALL.
`simular_monitoramento`	Procedimento	`lista_niveis`	`None`	Percorre a lista simulando ciclo de leitura em tempo real.

4. Código-Fonte Oficial (IraciBertoGomes_Ag11_DSI.py)

```
"""
Atividade: Controle de Níveis de Água - Agenda 11
Componente Curricular: Desenvolvimento de Sistemas I (DS I)
Curso Técnico em Desenvolvimento de Sistemas - ETEC / CPS
Estudante: Iraci Berto Gomes
Arquivo de Entrega: IraciBertoGomes_Ag11_DSI.py

Competência:
4. Elaborar sistemas aplicando princípios e paradigmas de programação.

Habilidades:
4.1 Codificar programas, utilizando técnica de programação estruturada.
4.2 Depurar e versionar programas, utilizando ambiente de desenvolvimento integrado.
"""

import os
import sys
import time
from colorama import init, Fore, Style, Back

# Configuração de suporte a UTF-8 no terminal Windows para acentuação perfeita
if sys.platform == 'win32':
try:
sys.stdout.reconfigure(encoding='utf-8')
except AttributeError:
pass

# Inicializa o colorama.
# autoreset=False permite demonstrar explicitamente o controle de restauração do padrão do terminal (Style.RESET_ALL)
init(autoreset=False)


# ======================================================================
# FUNÇÕES E PROCEDIMENTOS DO SISTEMA
# ======================================================================

def limpar_tela():
"""
Procedimento para limpar o terminal de forma multiplataforma (Windows e Linux/Mac).
Encapsulado em função para evitar efeitos colaterais na importação.
"""
os.system('cls' if os.name == 'nt' else 'clear')


def exibir_cabecalho():
"""
Procedimento que imprime o cabeçalho oficial do sistema no terminal.
"""
print(Style.BRIGHT + Fore.CYAN + "=" * 68)
print(" SISTEMA DE MONITORAMENTO DE NÍVEIS DE ÁGUA - RESERVATÓRIO")
print(" Telemetria e Controle de Risco Operacional")
print("=" * 68 + Style.RESET_ALL)
print()


def definir_cor_nivel(nivel):
"""
Função com Retorno (Responsabilidade Única - SRP):
Define e retorna o código de cor da biblioteca colorama correspondente
ao nível informado, seguindo rigorosamente a tabela de risco da atividade:

- Nível 1: Muito baixo (crítico) -> Vermelho (Fore.RED)
- Nível 2: Baixo -> Amarelo (Fore.YELLOW)
- Nível 3: Médio -> Verde (Fore.GREEN)
- Nível 4: Alto -> Ciano (Fore.CYAN)
- Nível 5: Muito alto (alerta) -> Azul (Fore.BLUE)

Parâmetros:
nivel (int): Nível de água do reservatório (1 a 5).

Retorno:
str: Código de cor ANSI da biblioteca colorama.
"""
if nivel == 1:
return Fore.RED
```

```
elif nivel == 2:
return Fore.YELLOW
elif nivel == 3:
return Fore.GREEN
elif nivel == 4:
return Fore.CYAN
elif nivel == 5:
return Fore.BLUE
else:
# Cor neutra de segurança caso seja informado um nível fora da escala
return Fore.WHITE


def gerar_barra_nivel(nivel, max_nivel=5):
"""
Função utilitária que gera uma barra visual ASCII para o medidor de nível.
Totalmente compatível com qualquer terminal Windows (sem caracteres não-mapeados).

Exemplo para nível 3: [###--] 60%
"""
cheio = "#" * nivel
vazio = "-" * (max_nivel - nivel)
porcentagem = (nivel / max_nivel) * 100
return f"[{cheio}{vazio}] {porcentagem:>3.0f}%"


def exibir_alerta_nivel(item_reservatorio):
"""
Procedimento responsável por formatar e imprimir na tela a situação
atual do reservatório com a cor correspondente e imediatamente
restaurar o estilo padrão do terminal.

Parâmetros:
item_reservatorio (dict): Dicionário contendo os dados do nível.
"""
nivel = item_reservatorio["nivel"]
situacao = item_reservatorio["situacao"]
detalhes = item_reservatorio["detalhes"]

# 1. Chamada da função que define a cor conforme o nível
cor = definir_cor_nivel(nivel)
barra = gerar_barra_nivel(nivel)

# 2. Exibição da mensagem no terminal com a cor correspondente
print(cor + Style.BRIGHT + f">> Nível {nivel}: {situacao:<22}" + Style.RESET_ALL, end=" ")
print(cor + f"Escala: {barra:<14}" + Style.RESET_ALL, end=" ")
print(Style.DIM + f"| Status: {detalhes}" + Style.RESET_ALL)

# 3. Restaura o estilo padrão do terminal após a exibição da mensagem
print(Style.RESET_ALL, end="")


def simular_monitoramento(lista_niveis, intervalo_segundos=0.6):
"""
Procedimento que simula o ciclo de monitoramento do reservatório,
percorrendo todos os níveis da lista e exibindo os alertas coloridos.

Parâmetros:
lista_niveis (list): Lista com os dicionários de cada nível.
intervalo_segundos (float): Tempo de espera entre cada leitura.
"""
print(Style.BRIGHT + "Iniciando ciclo de telemetria e diagnóstico dos sensores...\n" + Style.RESET_ALL)
print("-" * 68)

for item in lista_niveis:
exibir_alerta_nivel(item)
time.sleep(intervalo_segundos)

print("-" * 68)
print()


# ====================================================================
# PROGRAMA PRINCIPAL
# ====================================================================

if __name__ == "__main__":
```

```
# Limpa a tela para exibição limpa e profissional
limpar_tela()

# Exibe cabeçalho formatado
exibir_cabecalho()

# 1. Utilização de uma lista para armazenar os níveis e situações do reservatório
niveis_reservatorio = [
{
"nivel": 1,
"situacao": "Muito baixo (crítico)",
"detalhes": "Válvulas de emergência abertas. Risco iminente de desabastecimento."
},
{
"nivel": 2,
"situacao": "Baixo",
"detalhes": "Bomba de captação secundária acionada. Atenção para reposição."
},
{
"nivel": 3,
"situacao": "Médio",
"detalhes": "Capacidade estável. Volume ideal para consumo planejado."
},
{
"nivel": 4,
"situacao": "Alto",
"detalhes": "Reservatório com margem ampla. Entrada de água operando em 50%."
},
{
"nivel": 5,
"situacao": "Muito alto (alerta)",
"detalhes": "Capacidade máxima atingida! Vertedouro ativado contra transbordamento."
}
]

# 2. Executa a simulação de monitoramento
simular_monitoramento(niveis_reservatorio, intervalo_segundos=0.6)

# 3. Restauração explícita do estilo padrão do terminal (conforme orientação)
print(Style.RESET_ALL + Fore.RESET + Back.RESET, end="")
print(Fore.GREEN + Style.BRIGHT + "[OK] Ciclo de monitoramento concluido com sucesso!" + Style.RESET_ALL)
print("Estilo padrao do terminal restaurado com sucesso.\n")
```

5. Evidência de Execução Real no Terminal

6. Avaliação de Competências e Habilidades (ETEC / CPS)

• **Competência 4 (Elaborar sistemas aplicando princípios e paradigmas):** O projeto adota a programação estruturada como pilar, separando funções puras de cálculo e mapeamento da camada de saída e apresentação.
• **Habilidade 4.1 (Codificar programas com técnica estruturada):** Implementação de listas de dados ricas, laços de repetição com controle de fluxo, parametrização de funções e restauração rigorosa de cores ANSI.
• **Habilidade 4.2 (Depurar e versionar em IDE):** Testes executados com sucesso em ambiente integrado, tratamento de encoding UTF-8 no Windows e versionamento completo do projeto via Git/GitHub.